

TMC-103 – OPERATING SYSTEM

M.C.A. (First Semester) — Mid Semester Examination, Nov. 2022

Complete Solved Answer Key with Detailed Explanations

Reference: *Operating System Concepts*, 9th Edition — Abraham Silberschatz, Peter B. Galvin, Greg Gagne

Note: The original paper requires attempting **one sub-question [(a) OR (b)] from each of the 5 questions**, 10 marks each (Total = 50 marks). For complete revision of the syllabus, **both alternatives of every question are solved in full below**.

Q1. (a) Define OS. What are the functions and services of the operating system? Explain it with the help of a diagram.

[CO1] Marks: 10

Definition of Operating System

An Operating System (OS) is system software that acts as an intermediary between the computer user/application programs and the computer hardware. It manages hardware resources (CPU, memory, storage, I/O devices) and provides a convenient, efficient environment in which users can execute programs. Silberschatz defines an OS as "a program that manages the computer hardware" and also as a "resource allocator" and a "control program" that controls execution of user programs to prevent errors and improper use of the computer.

Goals of an OS

- Execute user programs and make solving user problems easier.
- Make the computer system convenient to use.
- Use computer hardware in an efficient manner.

Operating System: Interface between User, Applications and Hardware

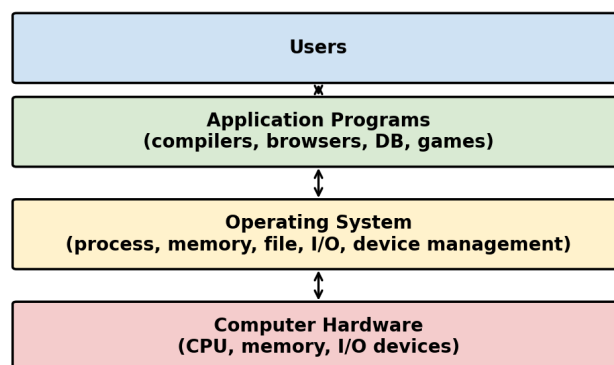


Fig. 1: Operating System as an interface between hardware, OS, applications and users

Functions of an Operating System

- **Process Management** – creating, scheduling, suspending, resuming and terminating processes; handling process synchronization and communication.
- **Memory Management** – keeping track of which parts of memory are in use, allocating and deallocating memory space, deciding which processes/data to move in and out of memory.

- **File Management** – creation/deletion of files and directories, mapping files onto secondary storage, backup of files.
- **I/O (Device) Management** – managing I/O devices through device drivers, buffering, spooling and providing a uniform interface to hardware.
- **Secondary Storage Management** – free-space management, storage allocation and disk scheduling.
- **Security and Protection** – protecting resources from unauthorized access and ensuring processes do not interfere with one another.
- **Networking** – managing communication and resource sharing in distributed/networked environments.
- **Command Interpretation** – providing a user interface (CLI/GUI) to interact with the system.

Services provided by an Operating System (as listed in Silberschatz Ch.2)

- **Program execution** – load a program into memory and run it, and be able to end execution (normally or abnormally).
- **I/O operations** – since user programs cannot directly execute I/O, the OS provides a way to perform I/O.
- **File-system manipulation** – read, write, create and delete files/directories, and manage permissions.
- **Communication** – exchange of information between processes, on the same computer or over a network, via shared memory or message passing.
- **Error detection** – the OS constantly needs to detect and correct errors from CPU, memory, I/O devices, or user programs.
- **Resource allocation** – allocating resources (CPU time, memory, files, I/O devices) to multiple users/jobs running concurrently.
- **Accounting** – keeping track of which users use how much and what kind of computer resources.
- **Protection and security** – ensuring controlled access to resources and authenticating users.

Together, these functions and services allow the OS to fulfil its two roles – providing a convenient abstraction of the hardware to users/programs, and efficiently managing and allocating the underlying hardware resources among competing requests.

Q1. (b) Write short notes on the following: Batch OS, Real-Time OS, Multitasking OS, Opensource OS, Commercial OS with one example of each.

[CO1] Marks: 10 (2 marks each)

1. Batch Operating System

In a Batch OS, similar jobs are grouped (batched) together and submitted to the computer as a group by an operator, without direct interaction with the user during execution. Jobs with similar requirements are executed together to improve throughput; the CPU processes one batch job at a time while a resident monitor automatically transfers control from one job to the next. There is no direct interaction between the user and the executing job, which reduces CPU idle time but increases turnaround time.

Example: IBM's early batch systems such as IBSYS / payroll processing systems.

2. Real-Time Operating System (RTOS)

An RTOS is designed to serve real-time applications that must respond to inputs/events within a strict, fixed time constraint (deadline). A *hard real-time system* guarantees that critical tasks complete on time (e.g., missing a deadline can cause system failure), while a *soft real-time system* gives higher priority to time-critical tasks but tolerates occasional missed deadlines. RTOS is characterised by minimal interrupt latency and thread priority-based preemptive scheduling.

Example: VxWorks, QNX, RTLinux – used in industrial control systems, robotics and satellite systems.

3. Multitasking (Time-Sharing) Operating System

A multitasking OS allows multiple programs (tasks) to run seemingly simultaneously on a single CPU by rapidly switching the CPU among them (time slicing/context switching), giving each process a small quantum of CPU time. This provides interactive, fast response time to multiple users/processes and is the logical extension of multiprogramming.

Example: Windows 11, Linux, macOS – where a user can browse the internet, play music and edit a document at the same time.

4. Opensource Operating System

An open-source OS is distributed along with its source code, which any user/developer is free to view, modify and redistribute, usually under a license such as GPL. This encourages community-driven development, transparency, security auditing and customization, and the software is typically available free of cost.

Example: Linux (and its distributions such as Ubuntu, Fedora), FreeBSD, Android (kernel).

5. Commercial (Proprietary) Operating System

A commercial OS is developed and owned by a company, distributed under a paid license; its source code is closed and users are not permitted to view or modify it. The vendor provides official support, regular updates and warranty, but customization by users is restricted.

Example: Microsoft Windows, Apple macOS/iOS.

Q2. (a) Differentiate between the following: (i) Multitasking and Multiprogramming (ii) Process and Threads

[CO1] Marks: 10 (5 marks each)

(i) Multitasking vs Multiprogramming

Basis	Multiprogramming	Multitasking
Concept	Keeps several jobs (programs) in main memory for execution. CPU always has one job to execute and switches rapidly between jobs.	Execution of multiple programs simultaneously, giving the illusion of simultaneous execution. CPU switches rapidly between jobs.
Objective	Maximize CPU utilization by minimizing idle time.	Provide fast response time and give the illusion of simultaneous execution.
User interaction	Little or no direct user interaction; jobs run in the background.	Highly interactive; user can work with multiple applications simultaneously.
Switching basis	Switches to another job when current job goes to the ready state.	Can be based on a preemptive or non-preemptive scheduling algorithm.
Example	Old batch systems running several jobs from a tape.	Modern desktop OS running browser, editor, music player, etc.

(ii) Process vs Threads

Basis	Process	Thread
Definition	A program in execution; an independent unit of execution.	A lightweight unit of execution within a process, also called a sub-process.
Memory	Has its own separate address space (code, data, stack, heap).	Threads of the same process share the code, data and heap memory.
Creation overhead	Heavyweight – creation/termination is slow.	Lightweight – creation/termination is fast.
Communication	Inter-process communication (IPC) is needed for communication between processes.	Threads of the same process can communicate directly.
Independence	Processes are independent of one another; termination of one process does not affect others.	Threads are not independent; termination of one thread affects others.
Context switch	Context switching between processes is expensive.	Context switching between threads of the same process is cheap.
Example	Running MS Word and a browser are two separate processes.	Typing and displaying text are two threads within the browser process.

Q2. (b) Differentiate between the following: (i) Microkernel and Monolithic kernel (ii) User mode and Kernel mode

[CO1] Marks: 10 (5 marks each)

(i) Microkernel vs Monolithic Kernel

Basis	Monolithic Kernel	Microkernel
Structure	Entire OS (process mgmt, memory mgmt, file system, device drivers, services (FS, Scheduling, etc.)) is in the kernel.	Only the most essential services (FS, Scheduling, etc.) are in the kernel, while other services are in user space.
Size	Large kernel image.	Small, minimal kernel.
Performance	Faster – components call each other directly within the kernel space.	Slightly slower – services in user space communicate via message passing over the kernel.
Reliability/Stability	A bug/crash in any component (e.g., a driver) can crash the whole system.	More robust – a bug in a user-space service doesn't crash the kernel.
Extensibility	Harder to extend/modify; recompilation of kernel often required.	Easier to extend – new services can be added as user-space programs.
Example	Traditional UNIX, Linux, MS-DOS.	Mach, QNX, Minix, (Windows NT partially hybrid).

(ii) User Mode vs Kernel Mode

Basis	User Mode	Kernel Mode
Definition	The restricted mode in which user applications run.	The privileged mode in which the operating system's core services run.
Access to hardware	No direct access to hardware; must request services via system calls.	Full access to all hardware and memory.
Mode bit	Mode bit = 1	Mode bit = 0
Instruction set	Can execute only a restricted (non-privileged) subset of instructions.	Can execute all instructions, including privileged instructions.
Switching	Switches to kernel mode via a trap/system call when a privileged operation is needed.	Switches to user mode via a return from system call.
Failure impact	A crash in a user-mode application typically affects only that process.	A crash in the kernel can bring down the entire system.
Example	Running a text editor or a web browser.	Handling an interrupt, executing a system call like read()/write().

Q3. (a) Consider the following processes with the length of CPU-burst time given in milliseconds. Apply Non-preemptive SJF scheduling for calculating the average waiting time, average turnaround time, CPU utilization and throughput.

[CO2] Marks: 10

Process	Arrival Time	Burst Time
P1	0	8
P2	1	4
P3	2	9
P4	3	5

Concept: Shortest Job First (SJF), non-preemptive: whenever the CPU becomes free, it is assigned to the process (among those that have already arrived) that has the smallest next CPU burst. Once started, a process runs to completion without being preempted.

Step 1: Determine execution order

- At $t = 0$, only P1 has arrived $\rightarrow$ P1 runs first (0–8).
- At $t = 8$, P2 (burst 4), P3 (burst 9) and P4 (burst 5) have all arrived $\rightarrow$ shortest is P2 $\rightarrow$ P2 runs (8–12).
- At $t = 12$, remaining are P3 (9) and P4 (5) $\rightarrow$ shortest is P4 $\rightarrow$ P4 runs (12–17).
- At $t = 17$, only P3 remains $\rightarrow$ P3 runs (17–26).

Gantt Chart — Non-preemptive SJF Scheduling

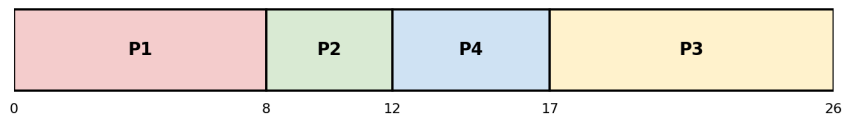


Fig. 2: Gantt chart for non-preemptive SJF scheduling

Step 2: Completion time, Turnaround time (TAT) and Waiting time (WT)

TAT = Completion Time – Arrival Time | WT = Turnaround Time – Burst Time

Process	Arrival	Burst	Completion	Turnaround (CT-AT)	Waiting (TAT-BT)
P1	0	8	8	$8-0 = 8$	$8-8 = 0$
P2	1	4	12	$12-1 = 11$	$11-4 = 7$
P3	2	9	26	$26-2 = 24$	$24-9 = 15$
P4	3	5	17	$17-3 = 14$	$14-5 = 9$

Step 3: Averages

- Average Turnaround Time = $(8+11+24+14)/4 = 57/4 = 14.25$ ms
- Average Waiting Time = $(0+7+15+9)/4 = 31/4 = 7.75$ ms

Step 4: CPU Utilization

Total time elapsed = 26 ms (0 to 26); Total CPU burst time = $8+4+9+5 = 26$ ms. Since the CPU is never idle (P1 starts exactly at the first arrival and every subsequent process is ready before the CPU frees up), CPU Utilization = $(\text{Total burst time} / \text{Total elapsed time}) \times 100 = (26/26) \times 100 = 100\%$.

Step 5: Throughput

Throughput = Number of processes completed / Total time = $4 / 26 = 0.1538$ processes per ms (≈ 1 process every 6.5 ms).

Q3. (b) Consider the following set of processes with the CPU burst time given in milliseconds. Draw a Gantt chart for a non-preemptive priority scheduling algorithm. Also, calculate average Turnaround time, average waiting time, CPU utilization and throughput.

[CO1, CO2] Marks: 10

Process	Burst Time	Priority
P1	10	3
P2	1	1
P3	2	4
P4	1	5
P5	5	2

Assumption (as in Silberschatz's convention): a smaller priority number indicates higher priority, i.e. Priority 1 is the highest priority. All processes are assumed to arrive at time 0.

Step 1: Order processes by priority (highest first)

P2 (priority 1) → P5 (priority 2) → P1 (priority 3) → P3 (priority 4) → P4 (priority 5)

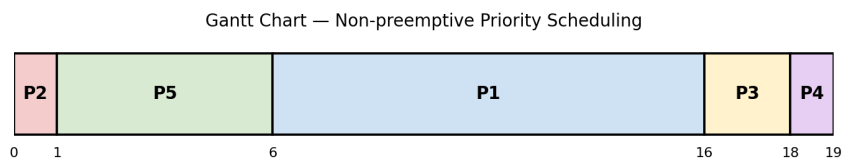


Fig. 3: Gantt chart for non-preemptive priority scheduling

Process	Burst	Priority	Completion	Turnaround (CT-0)	Waiting (TAT-BT)
P2	1	1	1	1	0
P5	5	2	6	6	1
P1	10	3	16	16	6
P3	2	4	18	18	16
P4	1	5	19	19	18

Averages

- Average Turnaround Time = $(1+6+16+18+19)/5 = 60/5 = 12 \text{ ms}$
- Average Waiting Time = $(0+1+6+16+18)/5 = 41/5 = 8.2 \text{ ms}$

CPU Utilization

Total elapsed time = 19 ms; total burst time = $10+1+2+1+5 = 19 \text{ ms}$ → CPU is busy throughout → CPU Utilization = $(19/19) \times 100 = 100\%$.

Throughput

Throughput = 5 processes / 19 ms = **0.263 processes per ms** (≈ 1 process every 3.8 ms).

Q4. (a) What is the purpose of the system call? Explain some system calls, with example. Also, differentiate between function and system call.

[CO2] Marks: 10

Purpose of a System Call

A system call is the programming interface through which a user program requests a service from the operating system's kernel – such as creating a file, allocating memory, or performing I/O. Since user programs run in user mode and cannot directly execute privileged instructions or access hardware, a system call provides a controlled, protected gateway (via a trap/software interrupt) into kernel mode so the OS can safely perform the requested operation on the program's behalf and then return control (and results) back to the user program. System calls are usually invoked indirectly through library functions/APIs (e.g., the C standard library) rather than being written directly by programmers.

Categories of System Calls (Silberschatz)

Category	Purpose	Examples
Process control	Create/terminate processes, load & execute programs, wait for events, abort()	<code>fork()</code> , <code>exec()</code> , <code>wait()</code> , <code>waitpid()</code> , <code>abort()</code>
File management	Create, delete, open, close, read, write files.	<code>open()</code> , <code>read()</code> , <code>write()</code> , <code>close()</code> , <code>create()</code>
Device management	Request/release a device, read/write to device.	<code>ioctl()</code> , <code>read()</code> , <code>write()</code>
Information maintenance	Get/set system data, time, date, process attributes.	<code>getpid()</code> , <code>alarm()</code> , <code>sleep()</code>
Communication	Send/receive messages, establish/end connections.	<code>pipe()</code> , <code>shmget()</code> , <code>mmap()</code>
Protection	Control access to resources, set permissions.	<code>chmod()</code> , <code>umask()</code> , <code>chown()</code>

Example – sequence for copying a file: a program uses `open()` to open the source file, `create()/open()` for the destination, then repeatedly calls `read()` and `write()` to transfer data, and finally `close()` on both files – each of these is a distinct system call that transfers control to the kernel.

Function (Library Call) vs System Call

Basis	Function Call (Library/User function)	System Call
Execution mode	Executes entirely in user mode, within the process address space.	Causes a switch from user mode to kernel mode to execute privileged instructions.
Who provides it	Written by the programmer or supplied by a user-level library.	Provided by the operating system kernel.
Overhead	Low overhead – simple procedure call with no mode switching.	High overhead due to mode switching (trap), context save/restore.
Purpose	Performs a general computation/task within the process.	Requests a service that only the OS/kernel can provide (I/O, memory allocation, etc.).
Example	A user-defined function <code>add(a,b)</code> , or a math library function like <code>sqrt()</code> .	System calls like <code>read()</code> , <code>write()</code> .

Q4. (b) What is a process? How it is different from Program? Draw and explain PCB in memory.

[CO2] Marks: 10

What is a Process?

A process is a *program in execution*. It is an active entity, and besides the program code (called the *text section*), it also includes the current activity as represented by the value of the program counter and the contents of the processor's registers. A process typically also includes the process stack (temporary data such as function parameters, return addresses, local variables), a data section (global variables) and a heap (memory dynamically allocated during run time).

Process vs Program

Basis	Program	Process
Nature	A passive entity – a set of instructions stored on disk (in a file, a program)	An active entity – a program that is currently being executed
State	Static; does not change unless edited.	Dynamic; goes through states like new, ready, running, waiting
Resources	Does not require CPU, memory or other resources while stored	Requires resources such as CPU time, memory, files and I/O
Lifespan	Exists permanently on secondary storage until deleted	Exists only for the duration of its execution; created and destroyed
Relationship	One program can be executed as multiple processes	Each running instance of a program is a separate process (e.g.,

Process Control Block (PCB)

Each process in an OS is represented in the kernel by a Process Control Block (also called a Task Control Block). The PCB is a data structure maintained by the OS in its memory that stores all the information needed to manage and later resume a process. It acts as the repository of process-specific information and serves as evidence that a process exists in the system.

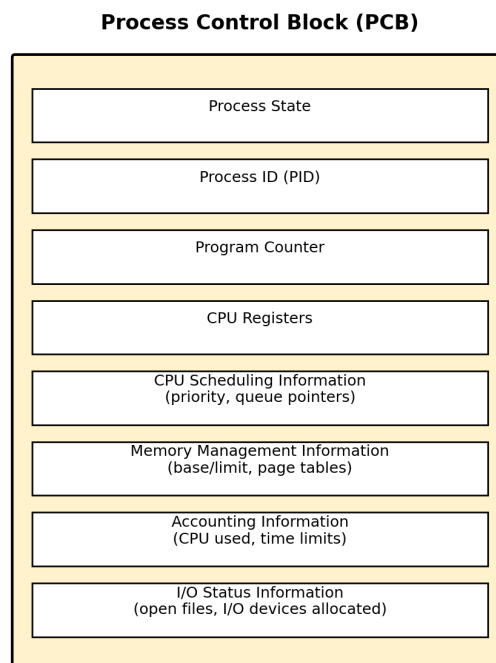


Fig. 4: Structure of a Process Control Block (PCB)

Fields of a PCB

- **Process state** – new, ready, running, waiting, terminated.
- **Process ID (PID)** – a unique identifier for the process (and pointer to its parent).
- **Program counter** – the address of the next instruction to be executed for this process.

- **CPU registers** – accumulators, index registers, stack pointers, general-purpose registers – saved when an interrupt occurs so execution can resume correctly later.
- **CPU scheduling information** – process priority, pointers to scheduling queues, and other scheduling parameters.
- **Memory management information** – value of base and limit registers, page tables or segment tables depending on the memory system used.
- **Accounting information** – amount of CPU and real time used, time limits, job/process numbers.
- **I/O status information** – list of I/O devices allocated to the process, list of open files.

When a context switch occurs, the OS saves the current state of the running process into its PCB and loads the saved state of the next process from its PCB into the CPU registers – this is how the PCB allows the OS to suspend and later correctly resume a process from exactly where it left off.

Q5. (a) Draw and explain the different states of processes. Explain CPU-I/O burst cycle.

[CO1, CO2] Marks: 10

Process States

As a process executes, it changes its *state*. Each process may be in one of the following states (Silberschatz, Ch.3):

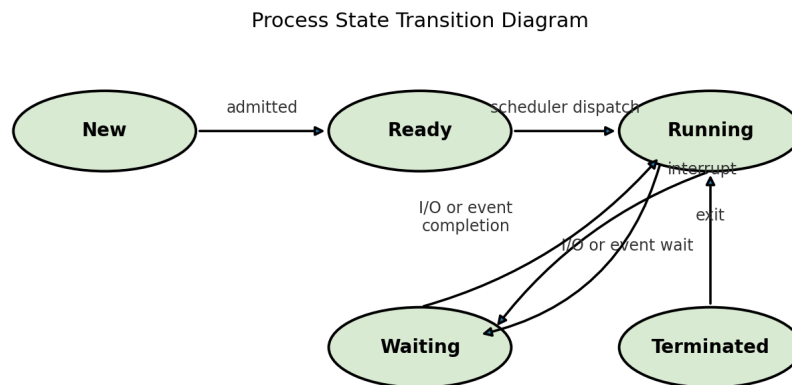


Fig. 5: Process State Transition Diagram

- **New** – the process is being created.
- **Ready** – the process is waiting to be assigned to a processor by the scheduler; it has all resources except the CPU.
- **Running** – instructions of the process are being executed by the CPU.
- **Waiting (Blocked)** – the process is waiting for some event to occur, such as completion of I/O or receipt of a signal.
- **Terminated** – the process has finished execution and is removed from the ready/running queues.

Transitions:

- New → Ready: process admitted by the long-term scheduler.
- Ready → Running: process selected (dispatched) by the short-term scheduler.
- Running → Waiting: process requests I/O or waits for an event.
- Waiting → Ready: I/O or event for which the process was waiting completes.
- Running → Ready: an interrupt occurs (e.g., time-slice expires) and the process is preempted.
- Running → Terminated: process finishes execution (or is aborted).

Note: only one process can be *running* on a given CPU core at any instant, though many processes may be *ready* or *waiting*.

CPU-I/O Burst Cycle

Process execution consists of a cycle of CPU execution and I/O wait. A process alternates between these two states: it begins with a *CPU burst* (a period during which it uses the CPU to execute instructions), followed by an *I/O burst* (a period during which it waits for an I/O operation such as disk read/write to complete), and this cycle repeats until the final CPU burst ends with a system request to terminate execution.

Alternating Sequence of CPU and I/O Bursts

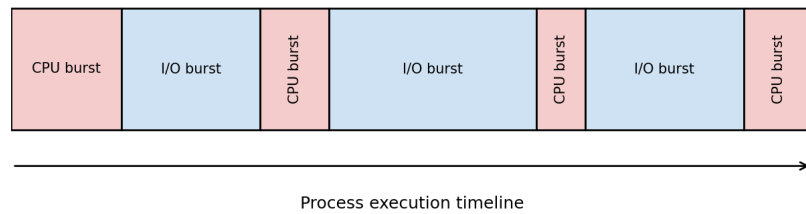


Fig. 6: Alternating sequence of CPU and I/O bursts of a process

CPU-bound processes have long CPU bursts and few I/O bursts (e.g., scientific computation), whereas I/O-bound processes have many short CPU bursts and frequent I/O bursts (e.g., interactive programs). This distinction is crucial for CPU scheduling algorithm design, since a good scheduling algorithm must balance the mix of CPU-bound and I/O-bound processes to maximize CPU utilization and system throughput.

Q5. (b) Explain the different types of schedulers. Also explain the queues which are all available during the process scheduling.

[CO1, CO2] Marks: 10

Types of Schedulers

A process migrates between the various scheduling queues throughout its lifetime; the OS must select processes from these queues using a scheduler. There are (up to) three types of schedulers:

- **Long-term Scheduler (Job Scheduler)** – selects processes from the job pool (on disk) and loads them into memory for execution, i.e., it controls the *degree of multiprogramming* (number of processes in memory). It executes infrequently (seconds/minutes) and can afford to be slow; it tries to maintain a good mix of CPU-bound and I/O-bound processes.
- **Short-term Scheduler (CPU Scheduler)** – selects, from among the processes in the ready queue, which process should be allocated the CPU next. It executes very frequently (every few milliseconds), so it must be extremely fast.
- **Medium-term Scheduler** – used in time-sharing systems to temporarily remove (swap out) a process from main memory to reduce the degree of multiprogramming and later swap it back in (swap out/in). It helps improve the process mix or free up memory.

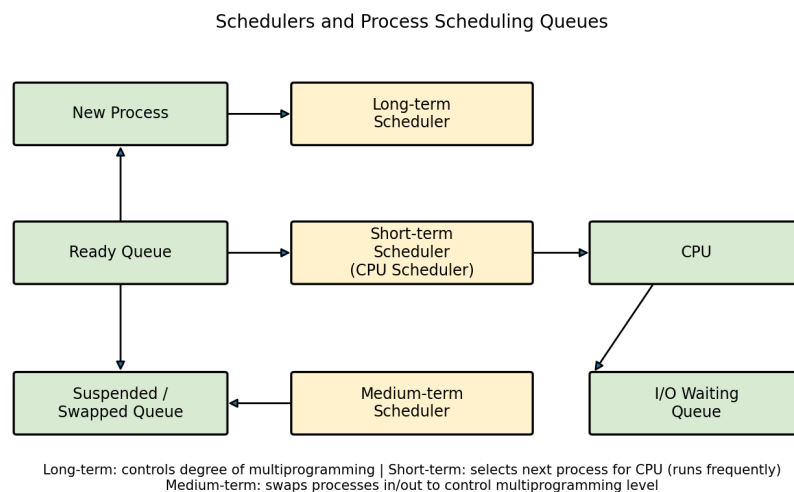


Fig. 7: Schedulers and process scheduling queues

Scheduling Queues

- **Job Queue** – contains all processes in the system, residing on the mass-storage device, waiting to be admitted into main memory.
- **Ready Queue** – contains all processes that reside in main memory and are ready and waiting to execute; generally maintained as a linked list, with a header pointing to the first and last PCBs in the list.
- **Device (I/O) Queues** – each device has its own queue; processes waiting for a particular I/O device are placed in that device's queue. A process may move between the ready queue and various device queues several times during its lifetime.

This overall movement can be represented by a *queueing diagram*: a new process first joins the ready queue; it waits there until it is selected for execution (dispatched); once running it may (a) issue an I/O request and be placed in an I/O queue, (b) create a new sub-process and wait for its termination, or (c) be removed forcibly as a result of an interrupt and be put back in the ready queue. This cycle continues until the process terminates, at which point it is removed from all queues and its PCB/resources are deallocated.

Prepared for MCA First Semester revision. Primary reference: Operating System Concepts, 9th Edition by Abraham Silberschatz, Peter B. Galvin and Greg Gagne (Wiley). Diagrams are original illustrations created for this answer key based on standard OS textbook concepts.